SOFTWARE REQUIREMENTS SPECIFICATION

Small Language Model for Interview Answer Evaluation

Feasibility-first planning baseline for a compact, self-controlled model

Document status	Baseline 1.0 - final planning document, pending owner approval
Prepared	12 October 2026
Requested workspace	C:\Users\Admin\Desktop\projects\model_traning
Primary constraints	Final model artifact under 300 MB; zero recurring hosting spend
Training intent	Train parameters from random initialization; no pretrained base weights
Product use	Formative feedback on student interview answers

Important feasibility statement

This document records the requested constraints, but does not claim they are jointly achievable. A sub-300 MB model trained from scratch is a research prototype target, not a presumed human-level evaluator. Student-facing use is prohibited until the evaluation and hosting gates in this SRS pass.

This is the planning boundary. Creating this PDF does not train a model, deploy a service, collect data, or modify the existing application.

1. Executive summary and planning decisions

The proposed system evaluates a student's interview response after speech has been transcribed to text. It should provide evidence-based practice feedback on relevance, factual/domain correctness, reasoning, completeness, clarity, and conclusion.

Why build it

- Reduce dependence on external generative-AI APIs and retain control over model weights, evaluation criteria, and deployment.
- Test whether a narrowly scoped model can provide useful feedback within a very small model artifact and a zero-cost hosting constraint.
- Keep model weights out of the student-facing browser and main application bundle.

Where it will live

- Training code, manifests, checkpoints, and reports are intended for the user's Desktop project folder named model_traning.
- The student app may remain on Vercel. Inference, if feasible, is a separate authenticated HTTPS service called by the app.
- No inference host is approved in this baseline; free hosting is an explicit feasibility gate.

Hard constraints

ID	Constraint	Interpretation
C-01	From scratch	Randomly initialized model parameters; do not substitute a pretrained foundation model without a written change approval.
C-02	Under 300 MB	Final inference artifact <=300 MB. Runtime RAM, tokenizer, cache, and service dependencies are measured separately.
C-03	Zero spend	No paid endpoint, subscription, or unapproved charge. If no free host passes, report NO-GO.
C-04	Student access	Must be remotely callable by authenticated app traffic; the developer laptop is not a production host.
C-05	Quality	No accuracy claim without a held-out, human-graded test set.

Planning decision: proceed only through a feasibility proof of concept. Do not represent the model as production-ready or use it for consequential student decisions unless all release gates pass.

2. Purpose, scope, users, and question types

Purpose

Specify behavior and verification criteria for a compact model that evaluates interview-answer text. This SRS is a planning baseline, not authorization to change application code.

In scope

- Question, optional role/topic context, answer transcript, rubric, structured evaluation, evidence, and uncertainty flags.
- From-scratch model research, data provenance, model evaluation, artifact size measurement, and no-cost hosting feasibility.

Out of scope

- Speech recognition, TTS, avatars, GD moderation, code execution, system-diagram vision, and LMS UI changes.
- Hiring/admissions decisions, high-stakes ranking, paid GPU/API usage, pretrained-base fine-tuning, and student-data collection without consent.

Users

Role	Need
Student	Understandable, constructive practice feedback tied to the question and their answer.
Instructor / rubric owner	Define scoring dimensions, examples, and acceptable evaluation behavior.
Project owner	Approve scope, data sources, spend, and release gates.
Model service	Accept authenticated text, return validated output, and avoid unnecessary data retention.

Supported interview question types

Type	Evaluation focus
Conceptual / factual	Correct explanation and direct relevance.
Applied / scenario	Application of concepts and justification of choices.
Reasoning / trade-off	Coherent assumptions, steps, alternatives, and trade-offs.
Behavioral	Relevant, specific evidence and a clear conclusion.
Code explanation	Accuracy of explanation for supplied code/approach; no execution implied.
System design	Components, connections, constraints, and trade-offs; no diagram vision implied.
Follow-up / challenge	Response to a specific earlier claim or clarification request.

Initial language scope: English text transcripts. Other languages, accents, and code-switching require a separate validation gate.

3. Functional requirements

P0 requirements are required for a proof of concept; P1 requirements are required before a student pilot; P2 requirements are later enhancements.

ID	Requirement
FR-01 / P0	Accept the interview question, optional role/topic, answer transcript, rubric version, and request identifier.
FR-02 / P0	Score relevance, factual/domain correctness, reasoning, completeness, clarity/organization, and conclusion on a documented 0-5 scale.
FR-03 / P0	Return overall and dimension scores, concise evidence, strengths, and actionable improvement suggestions.
FR-04 / P0	Tie each rationale to an exact transcript span where evidence exists; flag unsupported claims.
FR-05 / P0	Return a defined JSON schema; reject or safely retry invalid/incomplete output.
FR-06 / P0	Flag insufficient evidence for empty, unclear, off-topic, or out-of-scope answers.
FR-07 / P1	Accept a rubric version so results remain interpretable when criteria change.
FR-08 / P1	Do not infer protected traits, personality, or general intelligence from an interview response.
FR-09 / P1	Present feedback as practice guidance; keep progression decisions transparent and separate.
FR-10 / P1	Support rate limits, timeouts, health checks, and structured errors.
FR-11 / P2	Generate a concise follow-up question only after scoring behavior passes the release gate.

Minimum response fields

request_id, rubric_version, overall_score, dimension_scores, evidence_spans, strengths, improvements, uncertainty_flags, and model_version. Scores must not be shown without the rubric version.

4. Model, data, and training requirements

Training approach

- Start with randomly initialized parameters as requested. This requires a legally usable general text corpus before task examples can teach language behavior.
- Choose tokenizer, transformer size, context window, and training compute only after the feasibility gate defines a parameter budget.
- The 300 MB target applies to the final inference artifact, not training checkpoints or source corpus.
- Record corpus/license manifests, preprocessing, random seeds, training configuration, artifact hashes, and evaluation results.
- Do not train on student answers by default; training use requires explicit consent, lawful basis, and de-identification.

Data requirements

Split	Minimum planning requirement
Training	Licensed general text plus question/answer/rubric examples with human scores and evidence.
Validation	Separate examples for checkpoint selection; no overlap or near-duplicates with test.
Held-out test	At least 200 adjudicated examples for the first gate, balanced across question types and answer quality.
Human labels	Two independent reviewers; adjudicate material disagreement before creating consensus targets.
Coverage	Strong, average, weak, incomplete, incorrect, off-topic, concise, verbose, and ambiguous answers.

5. Data protection and deployment

Data protection

- Use only datasets with documented rights for research and intended deployment; keep a source and license register.
- Do not scrape private, copyrighted, or student records without authorization and documented rights.
- Minimize transcript retention; keep secrets out of repositories/logs; document deletion and access controls.
- Training and evaluation sets must be de-identified where possible and separated from production student sessions.

Logical deployment shape

Component	Responsibility	Placement
Student application	Collect transcript and display feedback.	Existing app host; never download model weights into browser.
API gateway	Authenticate, validate, rate-limit, call inference, validate response.	May remain on Vercel if it only brokers calls.
Inference runtime	Load model and generate structured evaluation.	Separate service; must pass memory, latency, privacy, and cost checks.
Training workspace	Code, data manifests, checkpoints, evaluation reports.	Requested folder: C:\Users\Admin\Desktop\projects\model_traning.

Hosting feasibility snapshot

Option	Verified constraint	Position in this SRS
Vercel Functions	Documented memory: 2 GB Hobby; up to 4 GB Pro/Enterprise. Package size is not RAM or GPU.	Use as app/API layer; do not assume it is a model host.
Render Free	Free web service: 0.1 CPU / 512 MB RAM; may sleep. Model file size is not total runtime memory.	Prototype test only; not approved for dependable student inference.
Hugging Face Spaces CPU Basic	Listed 2 vCPU / 16 GB; individual compute Spaces require a subscription; free CPU Spaces sleep after inactivity.	Not zero-cost for an individual without an existing subscription.
Developer computer	May run experiments if hardware permits.	Development only; not the public student endpoint.

Hosting go/no-go rule

Do not create paid services or incur charges without explicit approval. A free host must pass cold-start, sustained-inference, memory, latency, connectivity, and security tests. If no free host passes, report NO-GO and request a constraint change; do not silently substitute a paid provider.

Operational requirements

- HTTPS, server-side secrets, authenticated app-to-model calls, request limits, and abuse protection.
- Measure cold start, first token, full response, peak resident memory, crash rate, and sleep behavior.
- Provide health/model-version metadata; never expose internal traces or secrets.

Platform limits can change. Re-verify before any deployment.

6. Quality requirements and release gates

ID	Non-functional requirement
NFR-01	Final inference artifact <=300 MB, measured from deployed model files; report tokenizer/runtime separately.
NFR-02	Measure peak process memory using realistic prompt/context; do not infer RAM from file size.
NFR-03	No paid compute or API spend under the current constraint.
NFR-04	Encrypt student text in transit; minimize logs; define retention and deletion.
NFR-05	Record model/rubric versions; make evaluation reproducible from manifests.
NFR-06	Invalid output, low confidence, empty transcript, or timeout must fail safely, not invent a score.
NFR-07	Feedback is formative and is not the sole basis for high-impact student outcomes.

Proposed acceptance thresholds

These are gates to measure, not promises that training will achieve them:

- Held-out mean absolute score error <=1.0 on each core 0-5 dimension against adjudicated human scores.
- At least 80% of dimension scores within +/-1 point of human consensus; report each question type separately.
- At least 95% precision for evidence spans: each quoted span occurs in the answer and supports its feedback.
- At least 99% schema-valid responses; invalid output must fail safely.
- P95 response latency <=15 seconds on the proposed free host, with cold-start latency separately reported.
- No release if any category shows severe false-high scoring or reviewers find feedback misleading/harmful.

Evaluation procedure

- Freeze the held-out test set before final training; never tune on it.
- Report MAE, exact agreement, within-one-point agreement, per-dimension bias, and disagreement examples.
- Audit evidence, hallucinated claims, repeated-run variability, off-topic answers, and prompt injection in transcripts.
- Test resource use and expected concurrency, including wake-up after sleep and failure recovery.

Passing benchmark thresholds is necessary but not sufficient. The project owner must review results and explicitly authorize any student pilot.

7. Delivery plan, risks, and change control

Phase	Work	Estimate	Exit condition
0. Baseline	Accept this SRS; confirm folder spelling and constraints.	1 day	Frozen scope.
1. Feasibility	Verify data rights, compute, corpus, artifact definition, and host capacity.	3-5 days	Written GO/NO-GO; no paid resource.
2. Dataset	Create licensed corpus manifest, human-rated examples, splits, rubric.	1-3 weeks	Data audit and test set ready.
3. Scratch prototype	Implement small tokenizer/model training runs; measure behavior, file size, and resources.	1-3+ weeks	Reproducible baseline.
4. Evaluation	Run quality, safety, calibration, latency, memory tests; diagnose failure.	1-2 weeks	Gate report.
5. Hosting pilot	Only if free host and model pass; connect a test client.	3-5 days	Owner approval to proceed.

Indicative total: 4-10+ weeks to a research proof of concept if suitable data and compute are available. This is not a production estimate. A from-scratch model may fail quality or hosting gates; production readiness could take longer or require changing constraints.

Principal risks

Risk	Severity	Response
Tiny scratch model lacks language capability.	Critical	Research only; benchmark early; stop if quality misses gates.
No free host has sufficient capacity/latency.	Critical	Test before integration; no paid fallback without approval.
Insufficient or unlicensed text corpus.	High	Maintain rights register; exclude uncertain data.
Human ratings inconsistent.	High	Double-score, adjudicate, measure agreement.
Confident but incorrect feedback.	High	Evidence spans, uncertainty flags, safe failure, human review.
Student privacy/retention failure.	High	Minimize data, secure access, consent for training, deletion rules.
Overfit to narrow topics.	Medium	Hold out domains and test distribution shifts.

Change control

This SRS is the end of planning and the implementation baseline. Do not silently change the size limit, hosting budget, from-scratch requirement, data scope, or product use. Any change requires a written request describing rationale and impacts, followed by explicit project-owner approval. Until then, this baseline governs; creating this PDF makes no application-code changes.

8. Open gates, sign-off, and references

Open gates before implementation

Gate	Confirmation required
Workspace	Confirm destination spelling: C:\Users\Admin\Desktop\projects\model_traning.
Training corpus	Identify licensed text corpus and interview examples/rubrics available for train and test.
Compute	Identify available training hardware and prove the experiment can run without paid cloud resources.
Feasibility	Agree that failing quality or free-hosting gates is a valid outcome; do not inflate results.
Deployment	Identify a free, remotely accessible host that passes measured memory, latency, reliability, and security tests.

Definition of done for this SRS

Planning is complete when scope, constraints, question types, data rules, evaluation gates, hosting gate, risks, timeline, and change control are accepted. No model code, training run, deployment, or LMS change is authorized by this document alone.

Approval

Owner	Decision	Date
Project owner	Approve baseline / submit written change request	_____
Technical reviewer	Review feasibility gates and thresholds	_____

Glossary

Term	Definition
From scratch	Model parameters start from random initialization; pretrained weights are not used.
Artifact size	Size of deployed model files; distinct from runtime memory.
Quantization	Reducing weight precision to reduce storage/runtime memory, potentially affecting quality.
Held-out test set	Examples excluded from training/tuning and reserved for generalization measurement.
NO-GO	Documented stop where constraints cannot be met safely or credibly.

References - official platform documentation consulted 12 October 2026

1. Vercel, Vercel Functions Limits: vercel.com/docs/functions/limitations

2. Render, Compute Plans: render.com/docs/compute-plans

3. Render, Deploy for Free: render.com/docs/free

4. Hugging Face, Spaces Overview: huggingface.co/docs/hub/en/spaces-overview

5. Hugging Face, Manage Spaces: huggingface.co/docs/huggingface_hub/guides/manage-spaces

END OF SRS BASELINE 1.0